# Greedy Algorithms

***Questions and Answers***

## Question 1

### Problem Statement

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

**Input Format:** Take an integer from stdin.

**Output Format:** Print the integer which is change of the number.

**Explanation:** For input 64 we need a 50 Rs note, a 10 Rs note and two 2 rupee coins, so the output is 4.

### Example

| Input | Output |
|---|---|
| 64 | 4 |

### Answer (C program)

```
#include <stdio.h>

int main()
{
    int v, count = 0;
    int coins[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};

    scanf("%d", &v);

    for(int i = 0; i < 9; i++)
    {
        count = count + v / coins[i];
        v = v % coins[i];
    }

    printf("%d", count);

    return 0;
}
```

### Output of the program

| Input | Output |
|---|---|
| 64 | 4 |

## Greedy Approach

**1.** Keep the denominations in descending order: 1000, 500, 100, 50, 20, 10, 5, 2, 1.
**2.** For each denomination, add (V / coin) to the count.
**3.** Replace V with (V % coin) and continue with the next smaller denomination.

## Time Complexity

**Time complexity of this program: O(1)**
The loop runs over a fixed set of 9 denominations (O(d) for d denominations).

# Question 2

## Problem Statement

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

**Input Format:** First line: number of children n. Second line: n greed factors. Third line: number of cookies m. Fourth line: m cookie sizes.

**Output Format:** Print the maximum number of content children.

**Explanation:** You have 3 children and 2 cookies. The greed factors are 1, 2, 3. Both cookies have size 1, so only the child with greed factor 1 can be content. The output is 1.

**Constraints:** 1 <= g.length <= $3 * 10^4$  0 <= s.length <= $3 * 10^4$  1 <= g[i], s[j] <= $2^{31} - 1$

## Example

| Input | Output |
|---|---|
| 3<br>1 2 3<br>2<br>1 1 | 1 |

## Answer (C program)

```
#include <stdio.h>

int main()
{
    int n, m;
    scanf("%d", &n);

    int g[n];
    for(int i = 0; i < n; i++)
        scanf("%d", &g[i]);

    scanf("%d", &m);

    int s[m];
    for(int i = 0; i < m; i++)
        scanf("%d", &s[i]);

    int count = 0;

    for(int i = 0; i < n; i++)
    {
        for(int j = i + 1; j < n; j++)
        {
            if(g[i] > g[j])
            {
                int temp = g[i];
```

```
                g[i] = g[j];
                g[j] = temp;
            }
        }
    }

    for(int i = 0; i < m; i++)
    {
        for(int j = i + 1; j < m; j++)
        {
            if(s[i] > s[j])
            {
                int temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    int i = 0, j = 0;

    while(i < n && j < m)
    {
        if(s[j] >= g[i])
        {
            count++;
            i++;
        }
        j++;
    }

    printf("%d", count);

    return 0;
}
```

## Output of the program

| Input | Output |
|---|---|
| 3<br>1 2 3<br>2<br>1 1 | 1 |

## Greedy Approach

**1.** Sort the greed factors g[] and the cookie sizes s[] in ascending order.
**2.** Use two pointers i (child) and j (cookie), both starting at 0.
**3.** If s[j] >= g[i], give the cookie to the child: count++ and i++. In every case move to the next cookie: j++.
**4.** Stop when children or cookies run out; count is the answer.

## Time Complexity

**Time complexity of this program: $O(n^2 + m^2)$**
The two exchange sorts cost $O(n^2)$ and $O(m^2)$. The two-pointer matching is only O(n + m); with an efficient sort the total is O(n log n + m log m).

# Question 3

## Problem Statement

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories. If he has eaten i burgers with c calories each, then he has to run at least $3^i$ * c kilometers to burn out the calories. For example, if he ate 3 burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are ($3^0$ * 1) + ($3^1$ * 3) + ($3^2$ * 2) = 1 + 9 + 18 = 28.

But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm. Apply greedy approach to solve the problem.

**Input Format:** First line contains the number of burgers. Second line contains calories of each burger which is n space-separated integers.

**Output Format:** Print the minimum number of kilometers needed to run to burn out the calories.

**Sample:** Input: 3 / 5 10 7  Output: 76

**Another test:** Input: 3 / 1 3 2  Output: 18

## Example

| Input | Output |
|---|---|
| 3<br>5 10 7 | 76 |

## Answer (C program)

```
#include <stdio.h>
#include <math.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for(int i = 0; i < n; i++){
        scanf("%d",&arr[i]);
    }
    for(int i = 0; i < n; i++){
        for(int j = 0; j < n-1; j++){
            if(arr[j] < arr[j + 1]){
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    int s = 0;
    for(int i = 0; i < n; i++){
        s+= (pow(n, i))*arr[i];
    }
    printf("%d",s);
    return 0;
}
```

## Output of the program

| Input | Output |
|---|---|
| 3<br>5 10 7 | 76 |

## Greedy Approach

**1.** The multiplier grows with every burger eaten (position 0, 1, 2, ...).
**2.** To keep the total small, eat the burger with the highest calories first and the lowest calories last.
**3.** Sort the calories in descending order and add arr[i] times the multiplier for position i (the test cases use n as the base of the multiplier).

## Time Complexity

**Time complexity of this program: $O(n^2)$**
The bubble sort costs $O(n^2)$; the sum is O(n). With an efficient sort (merge sort / qsort) the total is O(n log n).

# Question 4

## Problem Statement

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N). Write an algorithm based on Greedy technique with a Complexity O(nlogn).

**Input Format:** First line specifies the number of elements - n. The next n lines contain the array elements.

**Output Format:** Maximum Array Sum to be printed.

## Example

| Input | Output |
|---|---|
| 5<br>2 5 3 4 0 | 40 |

## Answer (C program)

```
#include <stdio.h>

int main()
{
    int n, i, j, temp;
    int arr[100];
    int sum = 0;

    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }


    for(i = 0; i < n; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(arr[i] > arr[j])
            {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    for(i = 0; i < n; i++)
    {
        sum = sum + arr[i] * i;
    }

    printf("%d", sum);

    return 0;
}
```

## Output of the program

| Input | Output |
|---|---|
| 5<br>2 5 3 4 0 | 40 |

## Greedy Approach

**1.** The index i is the multiplier, so the largest values should sit at the largest indices.
**2.** Sort the array in ascending order.
**3.** Compute the sum of arr[i] * i.

## Time Complexity

**Time complexity of this program: $O(n^2)$**
The exchange sort costs $O(n^2)$; the sum is O(n). To meet the O(n log n) requirement in the question, use an O(n log n) sort such as merge sort or qsort.

# Question 5

## Problem Statement

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs (1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

**Input Format:** First line: n. Next n numbers: array A. Next n numbers: array B.

**Output Format:** Print the minimum sum of products.

## Example

| Input | Output |
|---|---|
| 3<br>1 2 3<br>4 5 6 | 28 |

## Answer (C program)

```
#include <stdio.h>

int main() {
    int n, i, j, temp, sum = 0;
    scanf("%d", &n);

    int a[n], b[n];

    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for(i = 0; i < n; i++)
        scanf("%d", &b[i]);

    for(i = 0; i < n; i++) {
        for(j = i + 1; j < n; j++) {
            if(a[i] > a[j]) {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }

            if(b[i] > b[j]) {
                temp = b[i];
                b[i] = b[j];
                b[j] = temp;
            }
        }
    }

    for(i = 0; i < n; i++)
        sum += a[i] * b[n - 1 - i];

    printf("%d", sum);

    return 0;
}
```

## Output of the program

| Input | Output |
| --- | --- |
| 3<br>1 2 3<br>4 5 6 | 28 |

## Greedy Approach

**1.** To minimize the sum of products, pair small values with large values.

**2.** Sort A in ascending order and B in ascending order.

**3.** Pair A[i] with B[n-1-i] and add the products.

## Time Complexity

**Time complexity of this program: $O(n^2)$**

The exchange sorts cost $O(n^2)$; the pairing sum is O(n). With an efficient sort the total is O(n log n).